

ITPROFEL2: OPEN SOURCE PROGRAMMING

Learning Module: Finals - Week 2

Instructor: Henry V. Dela Cruz

Topic: Advanced String Manipulation and Slicing

I. Overview and Objectives

In software development, data rarely arrives perfectly formatted. Much of a developer's time is spent parsing, cleaning, and extracting information from text. This week focuses entirely on strings: how they are stored in memory, how they behave, and how to surgically extract exact pieces of data from them.

Intended Learning Outcomes (ILOs):

- Define the importance of strings and explain String Immutability.
- Create strings using single, double, and triple quotes appropriately.
- Apply string operators to manipulate and evaluate text.
- Utilize the slice operator to extract substrings and use step slicing to skip characters.

II. Defining Strings and Immutability

A string in Python is a sequence of characters. It is a derived data type heavily utilized in nearly every application, from web forms to database queries.

1. Creating Strings

Python offers three ways to define string literals, allowing flexibility when your text contains quotes or spans multiple lines.

```
# Single and Double Quotes (functionally identical)
first_name = 'Henry'
last_name = "Dela Cruz"

# Triple Quotes (used for multi-line strings or docstrings)
sql_query = """
SELECT * FROM users
WHERE status = 'ACTIVE';
"""
```

2. String Immutability

A critical concept in Python is that **strings are immutable**. This means once a string is created in memory, it cannot be altered. If you perform an operation that seems to "change" a string, Python is actually destroying the old string and creating a completely new one in memory. This affects how we approach string manipulation for performance.

```
word = "Hello"
# word[0] = "Y" <-- This will cause a TypeError! Strings do not support item assignment.
word = "Y" + word[1:] # You must create a new string instead.
```

III. String Operators

Beyond mathematical applications, operators behave differently when applied to strings.

- **Concatenation (+):** Joins two strings together.
- **Repetition (*):** Repeats a string a specified number of times.
- **Membership (in / not in):** Checks if a specific substring exists within another string. Returns a Boolean.

```
print("Data" + "Base")      # Output: DataBase
print("-" * 10)             # Output: -----
print("dev" in "developer") # Output: True
```

IV. String Slicing and Substring Extraction

Every character in a string is assigned an index, starting at 0. Slicing allows you to grab a specific section of the string. The syntax is [start:stop:step].

1. Basic Slicing (Extracting Substrings)

The slice goes up to, but does **not** include, the stop index.

```
data_code = "ITPROFEL2-2026"

# Extracting a standard substring
subject = data_code[0:9]      # Grabs index 0 through 8
print(subject)               # Output: ITPROFEL2

# Omitting indices
prefix = data_code[:2]        # Start from beginning up to index 1 (Output: IT)
year = data_code[10:]         # Start at index 10 to the very end (Output: 2026)
```

2. Step Slicing (Skipping Characters)

The third parameter, step, determines how many characters to jump forward.

```
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

# Skip every other character
skip_letters = alphabet[0:10:2]
print(skip_letters)          # Output: ACEGI

# Reversing a string entirely using a negative step
reversed_string = alphabet[::-1]
print(reversed_string)       # Output: ZYXWVUTSRQPONMLKJIHGFEDCBA
```

Laboratory Coding Session: Institutional Data Parsing

Scenario: You are writing a script that receives raw, standardized data strings from a barcode scanner. The data format is fixed: [DEPT] - [YEAR] - [ID] - [STATUS]. You need to surgically extract this data using slice operators and print it in a human-readable format.

Instructions (Your code must implement these exact rules):

1. Declare a string variable named `raw_data` and assign it the value: "BSIT-2026-8492-ACTIVE"
2. Use standard slicing [`start:stop`] to extract the **Department** ("BSIT"). Store it in a variable.
3. Use standard slicing to extract the **Enrollment Year** ("2026"). Store it in a variable.
4. Use standard slicing to extract the **Student ID** ("8492"). Store it in a variable.
5. Use standard slicing to extract the **Account Status** ("ACTIVE"). Store it in a variable.
6. **Step Slicing Task:** Extract the Student ID again, but this time use step slicing with a negative step ([`: -1`]) to reverse the ID (it should become "2948"). Store it in a variable called `security_pin`.
7. Print the formatted output exactly as shown below.

Sample Program Run (Expected Output):

```
--- RAW DATA PARSER ---  
Raw String: BSIT-2026-8492-ACTIVE  
  
Extracted Department : BSIT  
Extracted Year       : 2026  
Extracted Student ID : 8492  
Extracted Status     : ACTIVE  
  
Generated Security PIN (Reversed ID): 2948
```

Submit your completed .py file to the instructor. Keep this file safe!